

项目仓库: eehubio/ezplm_smt_full

任务目标: 将当前纯静态 HTML 原型重构为可部署在 Vercel 的 Next.js 全栈交互系统, 并实现第一阶段可用的 RFQ、BOM 智能匹配、元器件数据查询、线上/线下采购比价及报价智能体。

不要继续直接修改现有独立 HTML 页面。当前 HTML 仅作为视觉参考, 必须移入 legacy-static 目录。

一、硬性约束

1. 使用当前稳定版 Next.js App Router、React、TypeScript。
2. 使用 Server Components + Client Components + Route Handlers。
3. 数据库使用 PostgreSQL, ORM 使用 Prisma。
4. 文件上传使用 Vercel Blob 或可替换的 FileStorageProvider。
5. AI 使用 Vercel AI SDK, 并通过 provider adapter 调用 Claude。
6. 所有外部 API Key 只能存在服务端环境变量中, 禁止暴露到浏览器。
7. 所有价格、GTB、Markup、PPV、人工费和总价必须由确定性 TypeScript 函数计算, 禁止由 LLM 直接计算。
8. AI 只能生成建议和草稿, 正式匹配、供应商选择、报价审批和 ERP 输出必须人工确认。
9. 每个写操作必须保存 tenantId、userId、时间和 AuditLog。
10. 应用必须既可部署到 Vercel, 也可通过 Next.js standalone Docker 镜像部署。
11. 不要在本阶段实现完整品质、PPAP、RMA、SN 追溯、高级 APS 或完整 ECN; 这些保留为 feature flag/backlog。
12. 不要通过爬虫访问 DigiKey 或 Mouser 网站, 只使用正式 API。

二、第一步: 整理仓库

1. 创建分支: feature/nextjs-agent-v1
2. 将现有 HTML/CSS/JS 移入: legacy-static/
3. 保留当前原型视觉风格、颜色、卡片和表格布局, 但将设计系统抽成:
4. app/globals.css
5. components/ui
6. components/shell
7. 创建 package.json、tsconfig.json、next.config.ts、eslint 配置和测试配置。
8. 创建基础路由:
9. /
10. /rfq
11. /bom
12. /bom/import
13. /bom/compare
14. /materials
15. /quotes
16. /procurement/rfq
17. /procurement/orders
18. /suppliers/opo
19. /reconciliation

20. /shortage
21. /kitting
22. /inventory
23. /settings
24. 所有详情页和子页面必须有返回上一层按钮。
25. 左侧菜单必须由统一 route config 生成，禁止每个页面复制一套导航。

完成后先提交 PR，不要直接进入 API 集成。

三、角色与工作台

实现角色：

- PM
- PROCUREMENT
- ENGINEERING
- MANAGEMENT
- SUPPLIER

实现四个内部工作台：

- PM 工作台
- 采购工作台
- 工程工作台
- 管理工作台

角色切换后必须同步更新：

- 当前用户名称/角色；
- 可见菜单；
- Dashboard KPI；
- 搜索范围；
- 可执行操作。

搜索范围：

- 工程页面默认只查 BOM、ECN/BOM变更记录、物料；
- 采购页面查供应商、采购 RFQ、采购 PO、OPO；
- PM 页面查 RFQ、客户、报价、BOM；
- 管理层可以全局查。

四、数据库模型

至少实现：

Tenant User Role UserRole Customer Supplier SupplierContact RFQ RFQAttachment RFQStatusHistory
BOM BOMVersion BOMLine BOMImportJob Part PartIdentifier CustomerPartMapping PartAlternate
InventorySnapshot OpenPOLine BomMatchCandidate BomLineDecision ExternalPartSnapshot
SupplierOffer PriceBreak Quote QuoteVersion QuoteLine QuoteApproval ProcurementRFQ SupplierQuote
SupplierQuoteLine PurchaseRequest OPOLine OPOLineReply ReminderLog AgentRun AgentStep
AgentEvidence AgentApproval AuditLog IntegrationJob ApiUsageLog

所有业务表必须带 tenantId。所有 update/delete 必须 tenant scoped。

五、RFQ 模块

RFQ 属于 PM 的“报价与客户”模块。

RFQ 支持：

- 创建 RFQ；
- 上传多个 BOM；
- 上传 Gerber、PDF、图片、工艺说明和其他附件；
- 保留原始客户文件；
- 标记附件类型；
- “不报价并关闭”按钮；
- 状态： DRAFT RECEIVED PARSING WAITING_ENGINEERING WAITING_PROCUREMENT QUOTING
PENDING_APPROVAL QUOTED CLOSED_NO_QUOTE LOST
- RFQ 可以关联多个 BOM；
- 显示上传人、客户、报价数量、截止时间、附件、处理记录；
- 预留 InboundEmailAdapter，但本阶段先不实现真实邮箱自动抓取。

六、BOM 导入与匹配

支持：

- CSV/XLSX；
- 图片/PDF；
- 多文件批量上传；
- 原始文件保存；
- 导入历史；
- 列映射；
- 非标准 BOM 转标准结构；
- 重复位号检测；
- EOL/禁用料检测；
- 封装不一致检测；
- 导入完成后可进入版本比对。

BOM 匹配顺序：

1. 客户料号映射；
2. 内部料号；
3. 精确 MPN；
4. Manufacturer + MPN；
5. 描述、规格、封装；
6. ezPLM 候选；
7. DigiKey/Mouser 候选；
8. 人工确认。

每个候选必须显示：

- 来源；
- 置信度；
- MFG；
- MPN；
- 封装；
- 生命周期；
- 内部库存；
- 呆滞数量；
- OPO；
- ETA；
- 价格；
- 替代关系；
- 数据更新时间。

七、ezPLM API Provider

不要猜测真实 API URL。

先定义：

```
interface EzplmPartsProvider { searchParts(input): Promise<CanonicalPart[]>; getPartByMpn(input):  
Promise<CanonicalPart | null>; batchResolve(inputs): Promise<BatchResolveResult[]>;  
getInventory(partIds): Promise<InventoryResult[]>; getCustomerMappings(customerId):  
Promise<CustomerPartMapping[]>; getAlternates(partId): Promise<AlternatePart[]>;  
getCompliance(partId): Promise<ComplianceResult>; }
```

实现：

- MockEzplmProvider
- HttpEzplmProvider
- Zod response validation
- API timeout
- retry
- circuit breaker
- contract tests

环境变量：

EZPLM_API_BASE_URL EZPLM_API_KEY

在没有真实文档和 Key 时使用 mock，不得在代码中伪造“已联调”。

八、DigiKey Provider

使用 Product Information V4。

实现：

DigiKeyAuthService DigiKeyTokenStore DigiKeyProvider

支持：

- ProductDetails
- ProductPricing
- KeywordSearch
- Substitutions
- RecommendedProducts

环境变量：

DIGIKEY_CLIENT_ID DIGIKEY_CLIENT_SECRET DIGIKEY_ACCOUNT_ID DIGIKEY_SITE DIGIKEY_LANGUAGE
DIGIKEY_CURRENCY

要求：

- OAuth Token 服务端缓存；
- 自动刷新；
- 相同 MPN 去重；
- Account ID 和 Locale Header 可配置；
- 429 重试；
- 记录 X-RateLimit；
- KeywordSearch 只用于候选，正式价格和库存使用 ProductDetails/ProductPricing；
- provider 不可用时返回 structured error，不阻断整个 BOM。

九、Mouser Provider

实现：

MouserProvider MouserRateLimiter

支持：

- partnumber
- keyword
- partnumberandmanufacturer
- keywordandmanufacturer

环境变量：

MOUSER_API_KEY MOUSER_API_BASE_URL

限制：

- 每次最多 50 结果；
- 每分钟限流；
- 每日配额；
- 对 429/5xx 退避；
- 返回缓存时间；
- 不允许浏览器直接调用 Mouser。

十、统一供应报价模型

所有 ezPLM、DigiKey、Mouser 和线下供应商报价统一为：

```
type NormalizedOffer = { provider; providerPartNumber; manufacturer; mpn; description; packaging;  
stock; moq; spq; leadTimeDays; lifecycle; rohs; reach; currency; priceBreaks; sourceUpdatedAt;  
sourceUrl; };
```

实现确定性函数：

```
getApplicablePriceBreak calculateRoundedPurchaseQty calculateExtendedPrice compareOffers  
rankOffers
```

排名不能只看最低单价，应考虑库存、需求数量、MOQ、SPQ、总金额、Lead Time、生命周期和供应商优先级。

十一、采购 RFQ

采购模块新增独立“采购 RFQ”：

- 接收 PM 的一个或多个 BOM；
- 保留原始客户 BOM；
- 导入线下供应商 Excel；
- 查询 ezPLM、DigiKey、Mouser；
- 展示全部供应商报价；
- 标识最低价格和推荐供应商；
- 采购可以选择、修改或拒绝推荐；
- 保存选择理由；
- 将采购报价反馈给 PM；
- 支持现货和期货模式；
- 更换货源必须记录新供应商、价格、币种、MOQ、SPQ、Lead Time 和报价时间。

十二、报价模拟

报价必须分成：

- 材料；
- 人工；
- NRE；
- SMT；
- DIP；
- 测试；
- 管理费；
- 其它费用。

报价行必须包含：

- 报价 MFG；
- 报价 MPN；
- 实际客户报价价格；
- Markup；
- 物料类别；
- 替代料 MFG/MPN；

- 采购成本；
- 最终物料报价；
- PPV。

实现多种人工费率模板并允许手工调整。

Quote 必须有版本和快照：

DRAFT PENDING_APPROVAL APPROVED REJECTED EXPIRED

提交审批后冻结快照。审批和正式 PDF/XLSX 必须使用快照。退回后创建新 Revision，不能覆盖已批准版本。

十三、智能体

使用 Vercel AI SDK 和 Claude，实现：

RfqIntakeAgent BomMatchingAgent SourcingAgent QuoteAgent OpoAgent

工具调用必须用 Zod schema。

读工具可自动执行。写工具必须展示确认卡片并由用户批准。

AgentRun 必须记录：

- 输入；
- 使用的工具；
- 外部数据证据；
- 模型输出；
- 人工确认；
- 最终写入结果；
- 失败信息；
- token/成本；
- 时间。

十四、OPO

建立行级 OPOLine 数据模型。

所有 KPI、未回复供应商表、差异表和异常表必须从同一数据源渲染。

供应商回复保存：

- replyEta；
- replyQty；
- replyNote；
- replyAt；
- replySource。

提醒任务：

- 每日 Cron 扫描 nextReminderAt；

- 提前四天催办；
- 使用 CRON_SECRET；
- 使用 idempotency key；
- 防止重复发送；
- 失败任务进入 IntegrationJob 重试；
- API 不可回写时生成 ERP Excel 模板。

十五、文件、缓存和批处理

文件使用 FileStorageProvider 抽象。

实现：

VercelBlobStorageProvider LocalStorageProvider（本地测试）

外部 API Cache Key：

provider + site + currency + mpn + manufacturer + quantity

建议 TTL：

- 价格/库存：15-30 分钟；
- 规格/生命周期/合规：24 小时；
- ezPLM 内部库存：按内部 API 策略。

超过 50 个唯一 MPN 时：

- 创建 ImportJob；
- 分批每次处理 10-20 个；
- 前端轮询或使用 Server-Sent Events 显示进度；
- 禁止在单个同步请求中处理整张大 BOM。

十六、页面反馈整改

必须实现：

- 所有子页面返回按钮；
- 角色化工作台；
- 管理层独立工作台；
- 管理层报价汇总、项目毛利、库存、呆滞和 DC Aging；
- Dashboard 风险卡片颜色；
- KPI 点击下钻；
- BOM 组合筛选；
- 时间筛选；
- 批量导出；
- BOM 导入历史；
- 报价页返回 RFQ；
- 采购和供应商协同单独菜单；
- AR 与 AP 通过角色和 Tab 区分；
- 搜索框按模块限定类型。

十七、测试

单元测试：

- MPN 标准化；
- GTB；
- MOQ/SPQ 圆整；
- 价格阶梯选择；
- Markup；
- PPV；
- 报价快照；
- Offer 排名；
- OPO 异常日期；
- tenant 隔离。

Playwright E2E：

1. PM 创建 RFQ 并上传多个附件；
2. RFQ 关闭为不报价；
3. BOM 导入、匹配、人工确认；
4. 同一 MPN 比较 ezPLM/DigiKey/Mouser/线下报价；
5. 采购选择供应商并反馈 PM；
6. 报价计算、审批、退回、新版本和批准；
7. OPO 回复、提醒和 ERP 导出；
8. 角色切换后菜单和权限变化；
9. API 失败时显示降级信息；
10. 所有关键操作生成 AuditLog。

十八、CI 与部署

每个 PR 必须运行：

`pnpm lint pnpm typecheck pnpm test pnpm test:e2e pnpm build prisma validate`

Vercel：

- GitHub Preview Deploy；
- Production 使用自定义域名；
- 环境变量区分 Preview / Production；
- Functions 与数据库优先测试 sin1；
- 自托管字体；
- 不在客户端暴露 Secret。

同时生成：

`Dockerfile docker-compose.local.yml .env.example`

十九、实施方式

不要一次性提交全部功能。

按以下 PR 拆分：

PR1 Next.js 基础工程和 UI Shell PR2 Auth/RBAC/Prisma/文件上传 PR3 ezPLM Provider PR4 DigiKey/Mouser Provider PR5 RFQ/BOM Agent PR6 采购 RFQ 和多源比价 PR7 Quote Agent 和审批 PR8 OPO/管理工作台 PR9 Playwright/部署/文档

每完成一个 PR：

1. 运行全部测试；
2. 给出 Vercel Preview URL；
3. 列出已完成和未完成；
4. 不得声称 mock API 已完成真实联调；
5. 等待人工确认后再进入下一个 PR。